

Trabajo Práctico 4 – Sistemas de Computación



Integrantes:

- Brigido Tomás
- Siñuka Javier
- Tosolini Agustín

1	Desarrollo	3
1.1	Compilación y preparación del módulo	3
1.2	Verificación de carga del módulo	4
1.3	Descarga del módulo y nueva verificación	4
1.4	Inspección de metadatos con modinfo	5
2	Respuestas	8
2.1	Desafío 1	8
2.1.1	¿Qué es checkinstall y para qué sirve?	8
2.2	Desafío 2	8
2.2.1	¿Qué funciones tiene disponibles un programa y qué funciones tiene disponibles un módulo?	8
2.2.2	Espacio de usuario y espacio del kernel	9
2.2.3	Espacio de datos	9
2.2.4	Drivers e investigación del contenido de /dev	9
2.3	Bibliografía	10

1 Desarrollo

En esta sección se documenta el procedimiento realizado para compilar, cargar, inspeccionar y descargar un módulo del kernel en Linux. La secuencia se expone de manera procedural, siguiendo el orden de ejecución observado en las capturas relevadas durante la práctica. El trabajo se realizó sobre el directorio correspondiente a la primera parte del proyecto de módulos, y posteriormente se emplearon herramientas de inspección del kernel para verificar tanto el estado de carga como los metadatos del módulo.

1.1 Compilación y preparación del módulo

En primer lugar, se ingresó al directorio de trabajo correspondiente a la primera parte del ejercicio y se ejecutó `make` sobre el módulo. Esta acción invocó el sistema de compilación del kernel para construir el objeto cargable `mimodulo.ko`. Tal como se verificó previamente, la compilación no presentó errores fatales: el proceso generó el artefacto esperado del módulo y dejó disponible el binario para su posterior inserción en el kernel.

Una vez finalizada la compilación, se procedió a cargar el módulo mediante `sudo insmod mimodulo.ko`. Dado que `insmod` no suele producir una salida descriptiva por sí mismo cuando la operación es exitosa, la validación de la carga se realizó consultando el buffer de mensajes del kernel con `sudo dmesg`.

```
1  cd part1/module
2  make
3  sudo insmod mimodulo.ko
4  sudo dmesg
```

La salida obtenida en `dmesg` evidenció tres aspectos relevantes: el kernel detectó la carga de un módulo externo (`out-of-tree`), informó que la verificación de firma no pudo completarse por ausencia de una clave requerida, y finalmente confirmó que el módulo fue cargado correctamente. Esto resulta coherente con una práctica académica en la que se trabaja con un módulo propio no firmado por una clave de confianza del sistema.

```

[70100.402025] pcieport 0000:00:1d.1: AER: Multiple Correctable error message received from 0000:05:00.0
[70100.402053] rtw88_8822ce 0000:05:00.0: PCIe Bus Error: severity=Correctable, type=Physical Layer, (Re
ceiver ID)
[70100.402061] rtw88_8822ce 0000:05:00.0: device [10ec:c822] error status/mask=00000001/0000e000
[70100.402070] rtw88_8822ce 0000:05:00.0: [ 0] RxErr (First)
[70110.718729] pcieport 0000:00:1d.1: AER: Multiple Correctable error message received from 0000:05:00.0
[70110.718778] rtw88_8822ce 0000:05:00.0: PCIe Bus Error: severity=Correctable, type=Physical Layer, (Re
ceiver ID)
[70110.718790] rtw88_8822ce 0000:05:00.0: device [10ec:c822] error status/mask=00000001/0000e000
[70110.718803] rtw88_8822ce 0000:05:00.0: [ 0] RxErr (First)
[70113.342984] mimodulo: loading out-of-tree module taints kernel.
[70113.342989] mimodulo: module verification failed: signature and/or required key missing - tainting ke
rnel
[70113.343536] Modulo cargado en el kernel.
(.venv) javier@javier-lcd: ~/Documents/University/Computer_Systems/TPs/TP4/files_project/kernel_modules/pa

```

1.1: Salida de dmesg luego de cargar mimodulo

1.2 Verificación de carga del módulo

Luego de la inserción, se utilizó `lsmod | grep mod` para verificar que el módulo efectivamente había quedado registrado entre los módulos actualmente cargados. La salida mostró la entrada correspondiente a `mimodulo`, junto con su tamaño en memoria y su contador de uso.

```
lsmod | grep mod
```

```

(.venv) javier@javier-lcd: ~/Documents/University/Computer_Systems/TPs/TP4/files_project/kernel_modules/pa
• rtl/module$ lsmod | grep mod
mimodulo                12288  0
(.venv) javier@javier-lcd: ~/Documents/University/Computer_Systems/TPs/TP4/files_project/kernel_modules/pa
○ rtl/module$

```

1.2: Verificación de mimodulo mediante lsmod

La presencia de `mimodulo` en esta lista confirmó que el kernel había aceptado la inserción del módulo y que este permanecía residente en memoria al momento de la consulta.

1.3 Descarga del módulo y nueva verificación

En una segunda etapa se procedió a retirar el módulo del kernel con `sudo rmmod mimodulo`. Una vez realizada la operación, se consultó nuevamente `dmesg` para observar el mensaje asociado a la descarga, y se volvió a ejecutar `lsmod | grep mod` para comprobar que el módulo ya no figuraba entre los módulos activos.

```

1  sudo rmmod mimodulo
2  sudo dmesg

```

La evidencia obtenida mostró el mensaje `Modulo descargado del kernel.` y, a continuación, la ausencia de `mimodulo` en la salida de `lsmod`, lo cual indica que la descarga se realizó de forma satisfactoria.

```
[70283.688808] rtw88_8822ce 0000:05:00.0: PCIe Bus Error: severity=Correctable, type=Physical Layer, (Re
ceiver ID)
[70283.688816] rtw88_8822ce 0000:05:00.0: device [10ec:c822] error status/mask=00000001/0000e000
[70283.688825] rtw88_8822ce 0000:05:00.0: [ 0] RxErr (First)
[70298.577039] pcieport 0000:00:1d.1: AER: Multiple Correctable error message received from 0000:05:00.0
[70298.577069] rtw88_8822ce 0000:05:00.0: PCIe Bus Error: severity=Correctable, type=Physical Layer, (Re
ceiver ID)
[70298.577077] rtw88_8822ce 0000:05:00.0: device [10ec:c822] error status/mask=00000001/0000e000
[70298.577086] rtw88_8822ce 0000:05:00.0: [ 0] RxErr (First)
[70307.093729] Modulo descargado del kernel.
(.venv) javier@javier-lcd:~/Documents/University/Computer_Systems/TPs/TP4/files_project/kernel-modules/pa
rtl/module$ lsmod | grep mod
(.venv) javier@javier-lcd:~/Documents/University/Computer_Systems/TPs/TP4/files_project/kernel-modules/pa
rtl/module$
```

1.3: Descarga del módulo y verificación posterior

Adicionalmente, se ejecutó el comando `cat /proc/modules | grep mod` desde la terminal. En este caso no se obtuvo ninguna salida. Este comportamiento es consistente con el estado del sistema luego de haber descargado el módulo, ya que `/proc/modules` enumera únicamente los módulos cargados en ese instante y el filtrado aplicado con `grep mod` no encontró coincidencias relevantes una vez removido `mimodulo`.

1.4 Inspección de metadatos con `modinfo`

Finalmente, se utilizaron consultas `modinfo` para inspeccionar los metadatos del módulo propio y compararlos con los de un módulo provisto por el sistema. En primer lugar, se consultó el archivo generado localmente:

```
modinfo mimodulo.ko
```

La salida permitió observar información característica del módulo compilado, incluyendo nombre, autor, descripción, licencia y la cadena `vermagic`, que expresa la compatibilidad del binario con la versión del kernel en ejecución.

1.4: Metadatos del módulo propio mediante modinfo

En segundo término, se inspeccionó un módulo perteneciente al árbol del sistema. Aunque en el esquema inicial la consulta se había pensado sobre una ruta terminada en `.ko`, en la práctica la ruta utilizada fue la que aparece en la captura, correspondiente al archivo comprimido del sistema:

```
modinfo /lib/modules/$(uname -r)/kernel/crypto/des_generic.ko.zst
```

Esta consulta permitió advertir una diferencia importante respecto del módulo desarrollado en la práctica: el módulo del sistema presenta metadatos adicionales vinculados con la firma criptográfica, tales como identificador de firma, firmante, clave de firma y bloque de firma. Esto refleja el mecanismo de integridad empleado por el kernel para módulos distribuidos oficialmente por el sistema.

```
(.venv) javier@javier-lcd:~/Documents/University/Computer_Systems/TPs/TP4/files_project/kernel-modules/pa
• rtl/module$ modinfo /lib/modules/$(uname -r)/kernel/crypto/des_generic.ko.zst
filename:          /lib/modules/6.17.0-23-generic/kernel/crypto/des_generic.ko.zst
alias:             crypto-des3_edc-generic
alias:             des3_edc-generic
alias:             crypto-des3_edc
alias:             des3_edc
alias:             crypto-des-generic
alias:             des-generic
alias:             crypto-des
alias:             des
author:            Dag Arne Osvik <da@osvik.no>
description:       DES & Triple DES EDE Cipher Algorithms
license:           GPL
srcversion:        A4E91C81A384FB1FA6D530F
depends:            libdes
intree:            Y
name:              des_generic
retpoline:         Y
vermagic:          6.17.0-23-generic SMP preempt mod_unload modversions
sig_id:            PKCS#7
signer:            Build time autogenerated kernel key
sig_key:           64:D7:DF:5C:E3:FA:59:21:38:42:72:F2:91:87:8E:AA:03:09:1A:12
sig_hashalgo:      sha512
signature:         AA:55:3E:C4:12:C8:3D:FB:A7:39:70:FD:A8:3A:4B:B5:29:56:B6:FC:
12:C4:AC:69:B5:43:76:05:BD:20:9F:98:BC:4F:87:4D:5D:D9:0C:8F:
38:4A:33:FC:FC:84:3D:35:BF:7B:11:79:52:88:AC:72:3D:0C:50:29:
80:C6:0C:84:23:29:87:EB:4D:65:52:64:03:D0:35:FB:E3:C9:69:37:
92:B4:AE:2F:9D:1B:9B:49:D0:16:A4:99:DA:68:22:C4:47:C9:BE:FF:
66:59:91:4B:E0:38:3C:06:41:6E:56:00:F7:30:36:54:26:FD:06:05:
F7:D4:24:72:C8:00:F8:88:5F:0C:89:04:A0:E3:CD:5E:16:F7:73:28:
96:BC:CF:31:E6:B7:41:50:75:18:97:1D:AC:A9:92:A9:C3:17:32:D3:
56:1F:EE:EA:FA:EF:92:AB:09:81:5D:A0:80:3F:E0:80:49:88:B4:4E:
BD:40:7B:AB:62:2C:02:08:06:68:20:52:5E:A2:3C:F0:20:59:CD:14:
95:5C:A6:BA:DB:87:C9:9D:1D:7A:12:6C:41:E3:5F:50:18:91:C4:1B:
A9:D7:B8:D2:B9:6E:C9:3C:E0:F6:5A:22:0E:94:DF:06:AB:94:32:8B:
C8:CD:8C:2D:70:2B:EE:7F:BD:5B:5C:C8:31:B6:D3:9A:53:0B:2B:AC:
A8:5B:E0:60:29:25:86:25:2B:42:22:28:01:8D:40:8A:0D:78:83:D8:
B3:99:C7:3C:FB:BC:2D:30:58:74:20:1C:F3:02:5F:93:29:B1:4E:EE:
BE:46:0F:B4:53:CA:5A:86:34:7B:72:F4:F7:6B:F8:26:D6:2A:5C:73:
3C:00:42:95:67:22:30:BF:75:FA:92:BE:89:80:EB:C8:DE:6A:2C:45:
F1:B3:F9:29:DE:A8:39:C3:DC:38:83:91:9D:79:42:6C:36:AD:A0:70:
EE:74:DF:9A:C3:01:4B:18:14:F4:DC:18:59:8C:FF:04:C9:19:6F:D9:
38:42:8B:01:D3:A0:8A:DA:F4:73:08:B1:9E:F1:ED:19:45:CB:AD:F4:
5E:52:EF:A5:39:27:26:3C:08:58:79:BE:AC:60:07:23:95:82:24:6F:
1D:56:2F:AC:B9:49:32:52:B0:39:6C:79:62:78:17:64:56:EC:9B:B7:
AA:76:93:28:BC:71:1C:EE:56:A4:B7:58:EF:DD:B9:16:C3:11:46:56:
13:3E:B2:1A:E2:90:5B:B0:8A:5F:15:A2:12:2C:56:CB:EA:EA:04:DE:
C5:69:69:D2:0D:59:19:A7:04:9E:FC:B0:4A:E4:31:18:06:A2:17:8F:
98:58:97:62:47:89:51:BB:67:11:50:CC

(.venv) javier@javier-lcd:~/Documents/University/Computer_Systems/TPs/TP4/files_project/kernel-modules/pa
• rtl/module$
```

1.5: Inspección de un módulo del sistema con modinfo

2 Respuestas

2.1 Desafío 1

2.1.1 ¿Qué es `checkinstall` y para qué sirve?

`checkinstall` es una herramienta orientada a sistemas GNU/Linux y otros entornos tipo Unix cuya finalidad es registrar una instalación realizada desde código fuente y transformarla en un paquete administrable por el sistema de paquetes de la distribución, típicamente `deb`, `rpm` o `tgz`. En lugar de ejecutar un `make install` convencional sin trazabilidad, `checkinstall` monitorea los archivos creados o modificados durante la instalación y construye, a partir de esa información, un paquete binario equivalente.

Su utilidad principal radica en que permite mantener control administrativo sobre software compilado manualmente. Esto facilita la posterior desinstalación, la auditoría de archivos instalados, la replicación de la instalación en otras máquinas compatibles y una mejor integración con las políticas de mantenimiento del sistema. En términos prácticos, constituye una solución intermedia entre la instalación informal desde fuentes y el empaquetado completo mantenido por la distribución.

2.2 Desafío 2

2.2.1 ¿Qué funciones tiene disponibles un programa y qué funciones tiene disponibles un módulo?

Un programa de espacio de usuario dispone, en primer término, de las funciones provistas por las bibliotecas de usuario, como la biblioteca estándar de C, y accede a servicios privilegiados por medio de llamadas al sistema. Por ello, operaciones como abrir archivos, reservar memoria dinámica, crear procesos o establecer comunicaciones de red se realizan a través de interfaces de usuario que luego son traducidas a syscalls hacia el kernel.

Un módulo del kernel, en cambio, no utiliza la biblioteca estándar como entorno de ejecución principal, sino que se integra directamente con la API interna del kernel. Sus funciones disponibles son aquellas que el núcleo exporta explícitamente mediante símbolos y subsistemas internos, tales como `printk`, `kmalloc`, `kfree`, `copy_to_user`, `copy_from_user`,

`request_irq`, o las rutinas de registro de dispositivos y controladores. En consecuencia, un módulo posee acceso a capacidades de mucho mayor privilegio, pero también opera bajo restricciones más severas: un error puede comprometer la estabilidad completa del sistema.

2.2.2 Espacio de usuario y espacio del kernel

El sistema operativo separa la ejecución en dos dominios fundamentales. El espacio de usuario es el ámbito donde corren las aplicaciones ordinarias, con privilegios limitados y aislamiento entre procesos. Allí, cada programa dispone de su propio espacio de direcciones virtuales y no puede acceder directamente ni al hardware ni a la memoria del kernel.

El espacio del kernel, por el contrario, es el dominio privilegiado desde el cual el sistema operativo administra memoria, interrupciones, planificación, dispositivos y políticas de protección. Los módulos cargables se ejecutan en este espacio y comparten el mismo contexto privilegiado del núcleo. La separación entre ambos dominios es esencial para la seguridad y la robustez del sistema, ya que impide que una falla en una aplicación común afecte de manera directa a la totalidad del sistema operativo.

2.2.3 Espacio de datos

En un programa de usuario, el espacio de datos forma parte de su espacio de memoria virtual e incluye, de manera general, datos globales inicializados, datos globales no inicializados, heap y stack. Este espacio pertenece al proceso y queda protegido por mecanismos de aislamiento; si el programa finaliza, su memoria es recuperada por el sistema.

En un módulo del kernel, en cambio, el espacio de datos no está aislado como el de un proceso ordinario, sino que queda incorporado al espacio global del kernel mientras el módulo permanezca cargado. Por ello, las estructuras de datos de un módulo deben diseñarse con especial cuidado respecto de concurrencia, sincronización y tiempo de vida. Una escritura inválida en ese contexto no compromete solamente al módulo, sino potencialmente al núcleo completo.

2.2.4 Drivers e investigación del contenido de `/dev`

Un driver es un componente de software, frecuentemente implementado como módulo del kernel, cuya responsabilidad es mediar entre el sistema operativo y un dispositivo físico o lógico. Su función consiste en traducir operaciones genéricas del sistema en acciones

concretas sobre el hardware o sobre una interfaz virtual. En Linux, muchos drivers se presentan al resto del sistema mediante archivos especiales de dispositivo.

El directorio `/dev` contiene precisamente esos archivos especiales, conocidos como nodos de dispositivo. No almacenan datos convencionales como un archivo común; su propósito es ofrecer una interfaz uniforme para interactuar con dispositivos y servicios del sistema. Dentro de `/dev` es habitual distinguir dos grandes clases:

- Dispositivos de carácter, que transfieren información como flujo secuencial de bytes, por ejemplo terminales, puertos serie o `/dev/null`.
- Dispositivos de bloque, que operan sobre bloques direccionables, como discos y particiones.

Cada entrada de `/dev` se asocia internamente con un número mayor (`major`) y un número menor (`minor`). El número mayor identifica el driver responsable de atender las operaciones sobre ese archivo de dispositivo, mientras que el menor permite distinguir instancias o subdispositivos administrados por ese mismo driver. En sistemas modernos, muchas de estas entradas son creadas dinámicamente por `udev`, en respuesta a los eventos de detección de hardware generados por el kernel.

Desde una perspectiva conceptual, `/dev` constituye el punto de encuentro entre el espacio de usuario y los controladores del kernel: una aplicación abre, lee, escribe o configura un nodo de dispositivo, y el kernel redirige dichas operaciones al driver correspondiente. Este esquema materializa uno de los principios centrales de Unix y Linux: tratar múltiples recursos del sistema mediante una interfaz homogénea basada en archivos.

2.3 Bibliografía

- Red Hat, *Restrictions Imposed by UEFI Secure Boot*:
<https://access.redhat.com/articles/1351013>
- Red Hat, *Managing kernel modules / signing kernel modules for secure boot*:
https://access.redhat.com/documentation/es-es/red_hat_enterprise_linux/8/html/managing_monitoring_and_updating_the_kernel/signing-kernel-modules-for-secure-boot_managing-kernel-modules
- The Linux Kernel Module Programming Guide:
<https://sysprog21.github.io/lkmpg/#what-is-a-kernel-module>
- Opensource.com, *Dynamic tracing in Linux user and kernel space*:
<https://opensource.com/article/17/7/dynamic-tracing-linux-user-and-kernel-space>